Error Handling Strategy

Version: 1.0

Status: Draft

Phase: 2 — Technical Design

Purpose

This document defines the conceptual strategy for handling errors during the execution of the Kommo ↔ Aloware integration.

Its purpose is to establish how execution failures are classified, isolated, and managed in order to maximize reliability while preventing unnecessary disruption to synchronization processes.

This document defines **how errors should be conceptually handled**, not the technical implementation of retries, logging, or monitoring.

Scope

This document defines:

- Error handling principles
- Error categories
- Error severity levels
- Failure response strategies
- Error isolation principles

This document intentionally excludes:

- Retry policies
- Logging implementation
- Monitoring configuration
- Alerting mechanisms
- Make error handlers
- API-specific behavior

Objectives

The error handling strategy has the following objectives:

- Preserve integration stability.
- Prevent data corruption.
- Isolate failures.
- Minimize operational disruption.
- Support recoverable processing.
- Facilitate troubleshooting.

Error Handling Principles

Fail Safely

When an error occurs, the current synchronization should stop safely without affecting unrelated executions.

No partial synchronization should leave the systems in an inconsistent state whenever avoidable.

Error Isolation

Failures should remain isolated to the affected synchronization event.

An error processing one lead, call, or SMS must never interrupt the processing of other independent events.

Preserve System Ownership

Errors must never alter the ownership model defined for the integration.

The authoritative system remains the source of truth regardless of synchronization failures.

Explicit Failure

Errors should never be silently ignored.

Every failure should produce a clear processing outcome.

Recoverability

Whenever possible, failures should be recoverable through controlled retry or operational intervention.

Permanent failures should not enter endless processing loops.

Error Categories

Validation Errors

Errors caused by invalid or incomplete business data.

Examples include:

- Missing required fields
- Invalid entity references
- Unsupported event types
- Missing mappings

These errors generally require correction before processing.

Business Rule Errors

Errors caused by violations of defined business rules.

Examples include:

- Invalid ownership
- Unauthorized synchronization
- Unsupported stage transition
- Assignment inconsistencies

These errors indicate that the event should not be processed.

Integration Errors

Errors occurring while communicating with external systems.

Examples include:

- API failures

- Authentication failures
- Network interruptions
- Service unavailability

These failures may be temporary or permanent.

Platform Errors

Errors generated by the orchestration platform itself.

Examples include:

- Workflow execution failures
- Internal processing exceptions
- Resource limitations
- Unexpected runtime failures

These errors affect execution rather than business logic.

Unexpected Errors

Errors that cannot be classified into known categories.

Examples include:

- Unknown exceptions
- Unexpected responses
- Unsupported platform changes
- Unhandled execution scenarios

These errors require investigation.

Error Severity

Errors are classified according to their operational impact.

Severity	Description
Low	Minor issue with limited operational impact.
Medium	Processing cannot continue for the current event but other flows remain unaffected.
High	Significant failure affecting one or more integration flows.
Critical	System-wide failure requiring immediate operational attention.

Severity classification guides operational response but does not determine implementation behavior.

Error Response Strategy

Depending on the error category, the integration should conceptually respond as follows.

Error Type	Expected Response
Validation Error	Reject event.
Business Rule Error	Reject event.
Temporary Integration Error	Suspend processing for later recovery.
Permanent Integration Error	Stop processing and require operational review.
Platform Error	Stop current execution.
Unexpected Error	Stop execution and classify for investigation.

Detailed recovery mechanisms are defined separately.

Failure Isolation

Failures should remain isolated at the smallest possible scope.

Examples include:

- Single lead
- Single contact
- Single communication event
- Single synchronization execution

Global failures should only occur when platform-wide dependencies become unavailable.

Error Propagation

Errors should not propagate unnecessarily between systems.

General principles include:

- Do not overwrite valid information after a failed synchronization.
- Do not trigger secondary failures.
- Do not generate cascading processing errors.
- Preserve the last known valid business state.

Error Recovery

Recovery depends on the nature of the error.

General principles include:

- Temporary failures may be recoverable.
- Permanent failures require corrective action.
- Invalid business data should be corrected before reprocessing.
- Successful events should never be reprocessed unnecessarily.

Detailed retry behavior is documented separately.

Design Constraints

The following constraints apply.

- Error handling must remain independent of business logic.
- Error handling must not modify business ownership.
- Error handling must remain deterministic.
- Failure processing should minimize operational impact.
- Recovery should not introduce duplicate business operations.

Design Principles

The strategy follows these principles:

- Reliability
- Isolation

- Recoverability
- Transparency
- Predictability
- Operational Stability
- Maintainability
- Scalability

Future Detailed Error Handling Specification

The implementation phase will define detailed operational specifications including:

- Error Code
- Error Category
- Severity
- Detection Method
- Processing Outcome
- Retry Eligibility
- Operational Action
- Escalation Policy
- Resolution Notes

This document serves as the conceptual foundation for those implementation procedures.

Related Documents

This document should be read together with:

- Validation Rules
- Retry Strategy
- Logging Strategy
- Monitoring Strategy
- Security Considerations
- Integration Flows Design

Retry Strategy

Version: 1.0

Status: Draft

Phase: 2 — Technical Design

Purpose

This document defines the conceptual strategy for retrying failed synchronization operations within the Kommo ↔ Aloware integration.

Its purpose is to establish when a failed operation should be retried, when it should be considered permanently failed, and how retries should be performed without compromising data integrity or creating duplicate business operations.

This document defines **when and why retries should occur**, not the technical implementation of retry mechanisms.

Scope

This document defines:

- Retry principles
- Retry eligibility
- Retry categories
- Retry constraints
- Retry decision model

This document intentionally excludes:

- Retry intervals
- Retry counters
- Make error handlers
- Scheduling configuration
- Logging implementation
- Monitoring implementation

Objectives

The retry strategy has the following objectives:

- Recover from temporary failures.
- Prevent unnecessary processing.
- Preserve data consistency.
- Avoid duplicate business actions.
- Reduce manual intervention.
- Improve integration reliability.

Retry Principles

Retry Only Recoverable Failures

Retries should only be performed when there is a reasonable expectation that the failure is temporary.

Failures that cannot succeed without external correction should not be retried.

Preserve Idempotency

Retrying an operation must not create duplicate business effects.

Repeated execution should produce the same business result as a single successful execution.

Independent Retry

Retries should be performed only for the failed synchronization event.

Other independent executions must continue normally.

Controlled Recovery

Retries should follow a controlled decision process.

Repeated failures should eventually stop and transition to operational review.

Retry Eligibility

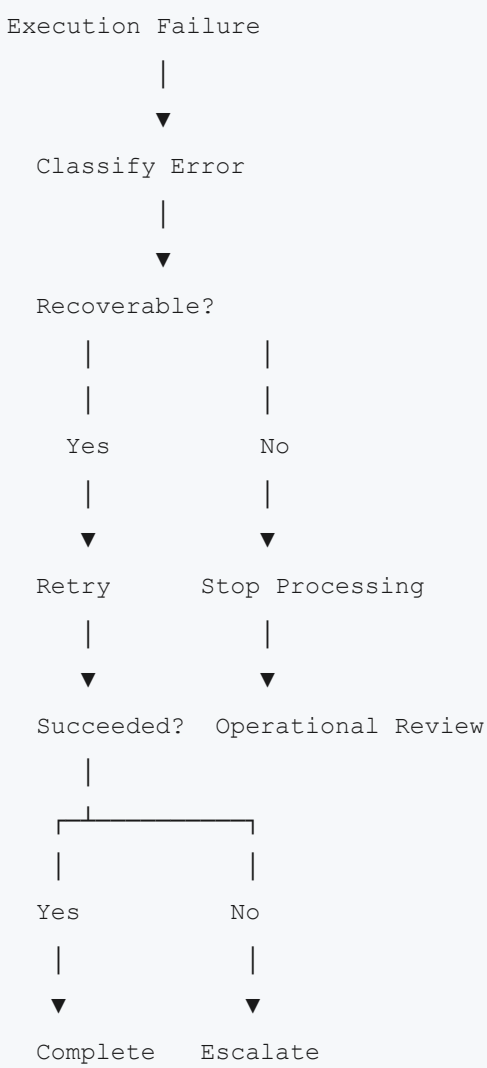
The following categories define whether a retry is conceptually appropriate.

Error Category	Retry Eligible
Validation Error	No
Business Rule Error	No
Authentication Error	No*
Authorization Error	No
Temporary Network Failure	Yes
Temporary API Failure	Yes
Service Unavailable	Yes
Timeout	Yes
Rate Limit	Yes
Unexpected Platform Failure	Depends on classification

*Authentication failures caused by expired credentials may become retryable after corrective action.

Retry Decision Model

Every failed execution should follow the same conceptual decision process.



Retry Categories

Immediate Retry

Used when the failure is expected to disappear quickly.

Examples include:

- Temporary network interruption
- Short-lived platform response issue

Deferred Retry

Used when recovery requires waiting for external conditions.

Examples include:

- Temporary service outage
- Rate limiting
- Planned maintenance

Manual Retry

Used when human intervention is required before processing can continue.

Examples include:

- Correcting business data
- Updating mappings
- Restoring credentials
- Resolving configuration issues

No Retry

Certain failures should never be retried automatically.

Examples include:

- Invalid payload
- Unsupported event
- Missing mandatory information
- Business rule violations

These events require correction rather than repeated execution.

Retry Constraints

The following constraints apply.

- Retries must not violate data ownership.
- Retries must preserve idempotency.
- Retries must remain deterministic.
- Retries should never create infinite processing loops.
- Retries should not generate unnecessary API traffic.

Retry Termination

Retry processing should eventually terminate.

Processing should stop when:

- The operation succeeds.
- The failure is classified as permanent.
- Operational review becomes necessary.
- Recovery is no longer possible.

The integration should avoid endless retry cycles.

Retry Relationship with Error Handling

Error handling determines the nature of the failure.

Retry determines whether recovery should be attempted.

The two processes are complementary.

```
Execution Failure
          |
          ▼
  Error Classification
          |
          ▼
  Retry Decision
          |
          ▼
  Recovery Attempt
          |
          ▼
  Processing Outcome
```

Retry Relationship with Logging

Every retry attempt should generate operational records.

The retry process should be traceable throughout its lifecycle.

Logging requirements are defined separately.

Retry Relationship with Monitoring

Retry activity contributes to operational monitoring.

Repeated retry failures may indicate:

- Platform instability
- Configuration issues
- Authentication problems
- Business data quality issues

Monitoring requirements are defined separately.

Design Principles

The retry strategy follows these principles:

- Recoverability
- Reliability
- Idempotency
- Controlled Processing
- Failure Isolation
- Predictability
- Operational Efficiency
- Scalability

Future Detailed Retry Specification

The implementation phase will define detailed retry specifications including:

- Retry Policy ID
- Eligible Error Types
- Maximum Retry Attempts
- Retry Interval
- Backoff Strategy
- Escalation Threshold
- Final Processing Outcome

- Operational Notes

This document serves as the conceptual foundation for those implementation procedures.

Related Documents

This document should be read together with:

- Error Handling Strategy
- Logging Strategy
- Monitoring Strategy
- Validation Rules
- Integration Flows Design

Logging Strategy

Version: 1.0

Status: Draft

Phase: 2 — Technical Design

Purpose

This document defines the conceptual logging strategy for the Kommo ↔ Aloware integration.

Its purpose is to establish what operational information should be recorded during synchronization processes to support traceability, troubleshooting, auditing, and operational analysis.

This document defines **what should be logged**, not **how logs are stored or implemented**.

Scope

This document defines:

- Logging principles
- Logging objectives
- Log categories
- Log contents
- Logging constraints

This document intentionally excludes:

- Log storage technologies
- Log retention policies
- Monitoring dashboards
- Alerting mechanisms
- Platform-specific logging implementation
- Make execution logs

Objectives

The logging strategy has the following objectives:

- Provide end-to-end traceability.
- Support troubleshooting.

- Facilitate operational auditing.
- Improve incident investigation.
- Enable performance analysis.
- Support future monitoring capabilities.

Logging Principles

Record Meaningful Events

Only events that contribute to understanding the behavior of the integration should be recorded.

Logging should prioritize operational value over volume.

Preserve Traceability

Every synchronization should be traceable from its origin to its final outcome.

Related events should be linked through common identifiers whenever possible.

Remain Non-Intrusive

Logging must never alter business behavior or synchronization results.

Logging exists to observe execution, not influence it.

Consistency

Similar events should produce similar log records.

Consistent logging simplifies troubleshooting and operational analysis.

Logging Categories

Execution Logs

Record the lifecycle of synchronization executions.

Examples include:

- Execution started
- Processing completed
- Execution failed
- Execution cancelled

Validation Logs

Record validation outcomes before synchronization.

Examples include:

- Validation passed
- Validation rejected
- Missing required information
- Invalid mapping

Synchronization Logs

Record synchronization activities performed between systems.

Examples include:

- Entity synchronized
- Record updated
- Record created
- Synchronization skipped

Error Logs

Record failures encountered during execution.

Examples include:

- API failure
- Timeout
- Authentication failure
- Business rule violation

Detailed error handling is defined separately.

Retry Logs

Record retry-related activities.

Examples include:

- Retry scheduled
- Retry started
- Retry succeeded
- Retry exhausted

Retry behavior is defined separately.

Log Contents

Each log entry should conceptually include information such as:

- Timestamp
- Event type
- Execution identifier
- Correlation identifier
- Source system
- Destination system
- Entity type
- Processing status
- Severity
- Summary message

Additional implementation-specific information may be recorded when appropriate.

Log Levels

The integration may classify logs according to their operational importance.

Level	Purpose
Debug	Detailed execution information for development and troubleshooting.
Information	Normal processing events.
Warning	Recoverable situations requiring attention.
Error	Processing failures affecting execution.
Critical	Severe failures requiring immediate operational response.

The implementation may adapt these levels as needed.

Correlation and Traceability

Every synchronization execution should be traceable across all processing stages.

Conceptually, logs should allow operators to reconstruct:

- Event origin
- Validation outcome
- Transformation process
- Synchronization result
- Retry attempts
- Final execution status

This enables complete operational visibility.

Sensitive Information

Logs should avoid exposing sensitive business information unnecessarily.

General principles include:

- Protect authentication credentials.
- Avoid recording confidential information unless operationally required.
- Minimize exposure of personally identifiable information.
- Record only information necessary for diagnosis and auditing.

Security requirements are defined separately.

Logging Constraints

The following constraints apply.

- Logging must not modify business data.
- Logging must remain independent of business logic.
- Logging should not introduce duplicate processing.
- Logging should support deterministic execution analysis.
- Logging should remain consistent across all integration flows.

Relationship with Monitoring

Logging provides the operational information consumed by monitoring processes.

Monitoring analyzes log information to identify:

- Processing failures
- Execution trends
- Operational anomalies
- System health indicators

Monitoring behavior is defined separately.

Relationship with Auditing

Logs provide historical evidence of synchronization activities.

They support:

- Operational reviews
- Incident investigations
- Compliance verification
- Business traceability

Logging should therefore remain complete and consistent.

Design Principles

The logging strategy follows these principles:

- Traceability
- Transparency
- Consistency
- Simplicity
- Operational Visibility
- Reliability
- Maintainability
- Scalability

Future Detailed Logging Specification

The implementation phase will define detailed logging specifications including:

- Log Event ID
- Event Category
- Log Level
- Timestamp Format
- Correlation ID
- Execution ID
- Entity Identifier
- Message Template
- Additional Metadata
- Storage Requirements

This document serves as the conceptual foundation for those implementation specifications.

Related Documents

This document should be read together with:

- Error Handling Strategy
- Retry Strategy
- Monitoring Strategy
- Security Considerations
- Integration Flows Design

Monitoring Strategy

Version: 1.0

Status: Draft

Phase: 2 — Technical Design

Purpose

This document defines the conceptual monitoring strategy for the Kommo ↔ Aloware integration.

Its purpose is to establish how the operational health of the integration should be observed, evaluated, and assessed throughout its lifecycle.

Monitoring provides continuous visibility into synchronization activities, enabling early detection of operational issues and supporting long-term reliability.

This document defines **what should be monitored**, not **how monitoring is implemented**.

Scope

This document defines:

- Monitoring principles
- Monitoring objectives
- Monitoring categories
- Health indicators
- Operational visibility

This document intentionally excludes:

- Monitoring platforms
- Dashboards
- Alerting mechanisms
- Log storage
- Metrics implementation
- Infrastructure monitoring

Objectives

The monitoring strategy has the following objectives:

- Maintain operational visibility.
- Detect integration issues early.
- Measure synchronization health.
- Support operational decision-making.
- Improve reliability over time.
- Enable proactive maintenance.

Monitoring Principles

Continuous Observation

The integration should be continuously observed throughout its operation.

Monitoring should provide visibility into the current operational state without interfering with processing.

Operational Health

Monitoring focuses on the operational condition of the integration rather than individual execution details.

Its objective is to identify trends, anomalies, and degradation over time.

Early Detection

Potential issues should be identified as early as possible to minimize operational impact.

Monitoring should prioritize timely detection over reactive investigation.

Independent Observation

Monitoring should remain independent of business processing.

The failure of monitoring should never interrupt synchronization activities.

Monitoring Categories

Execution Monitoring

Observes synchronization execution activity.

Examples include:

- Executions started
- Executions completed
- Executions failed
- Executions pending

Performance Monitoring

Observes the operational performance of synchronization processes.

Examples include:

- Processing duration
- Response times
- Execution throughput
- Processing latency

Error Monitoring

Observes execution failures.

Examples include:

- Error frequency
- Error categories
- Failure trends
- Critical failures

Error handling is defined separately.

Retry Monitoring

Observes retry behavior.

Examples include:

- Retry frequency
- Retry success rate
- Retry exhaustion
- Recoverable failures

Retry policies are defined separately.

Data Quality Monitoring

Observes the quality of synchronized information.

Examples include:

- Validation failures
- Mapping failures
- Unsupported events
- Rejected synchronizations

Health Indicators

The operational health of the integration may be evaluated using indicators such as:

- Successful synchronization rate
- Failed synchronization rate
- Average processing time
- Retry rate
- Validation rejection rate
- API availability
- Integration availability

These indicators provide a conceptual view of integration health.

Operational States

The monitoring model recognizes the following conceptual operational states.

State	Description
Healthy	The integration operates within expected parameters.
Degraded	Processing continues but operational performance is reduced.
Unstable	Failures occur frequently and reliability is impacted.
Unavailable	Synchronization cannot be performed.

The criteria defining these states are specified during implementation.

Operational Visibility

Monitoring should provide visibility into:

- Current operational status
- Synchronization activity
- Failure distribution
- Processing performance
- Historical execution trends
- Overall integration health

This information supports operational management without exposing implementation details.

Relationship with Logging

Monitoring consumes operational information produced by the logging strategy.

Logs provide execution records.

Monitoring transforms those records into operational insights.

Relationship with Error Handling

Monitoring observes the occurrence and evolution of errors.

Error handling determines how failures are processed.

Monitoring evaluates their operational impact.

Relationship with Retry

Monitoring evaluates the effectiveness of retry mechanisms.

Examples include:

- Retry success trends
- Increasing retry frequency
- Persistent recovery failures
- Operational degradation

Retry execution is defined separately.

Monitoring Constraints

The following constraints apply.

- Monitoring must remain independent of synchronization logic.
- Monitoring must not modify business data.
- Monitoring must not interfere with processing.
- Monitoring should support long-term operational analysis.
- Monitoring should remain consistent across all integration flows.

Design Principles

The monitoring strategy follows these principles:

- Visibility
- Reliability
- Transparency
- Proactive Observation
- Operational Awareness
- Consistency
- Scalability
- Maintainability

Future Detailed Monitoring Specification

The implementation phase will define detailed monitoring specifications including:

- Health Metrics
- Key Performance Indicators (KPIs)
- Service Level Indicators (SLIs)
- Monitoring Thresholds
- Dashboard Definitions
- Operational Reports
- Trend Analysis
- Escalation Conditions
- Observation Windows

This document serves as the conceptual foundation for those implementation specifications.

Related Documents

This document should be read together with:

- Logging Strategy
- Retry Strategy
- Error Handling Strategy
- Security Considerations
- Integration Flows Design

Security Considerations

Version: 1.0

Status: Draft

Phase: 2 — Technical Design

Purpose

This document defines the conceptual security principles that govern the Kommo ↔ Aloware integration.

Its purpose is to ensure that data exchanged between systems is protected throughout the integration lifecycle while maintaining confidentiality, integrity, availability, and accountability.

This document defines **security principles and considerations**, not platform-specific security configurations.

Scope

This document defines:

- Security principles
- Authentication considerations
- Authorization principles
- Data protection
- Operational security
- Security responsibilities

This document intentionally excludes:

- API key configuration
- OAuth implementation
- Encryption algorithms
- Firewall configuration
- Infrastructure security
- Vendor-specific security settings

Objectives

The security model has the following objectives:

- Protect business information.
- Prevent unauthorized access.
- Preserve data integrity.
- Ensure secure communication.
- Support operational accountability.
- Reduce security risks.

Security Principles

Least Privilege

Every system, integration component, and credential should operate with only the permissions required to perform its intended function.

Unnecessary permissions should be avoided.

Defense in Depth

Security should be applied through multiple complementary layers rather than relying on a single protection mechanism.

Each layer contributes to reducing overall risk.

Secure by Default

Security should be considered the default operational state.

Any optional functionality should require explicit configuration rather than reducing security automatically.

Separation of Responsibilities

Business ownership, operational processing, and security responsibilities should remain clearly separated.

No single integration component should assume unnecessary control over the entire synchronization process.

Authentication Considerations

All communication between systems should occur through authenticated requests.

Authentication credentials should:

- Be unique for each integration.
- Be securely managed.
- Remain confidential.
- Be replaceable when necessary.
- Never be exposed through operational processes.

Authentication mechanisms are defined during implementation.

Authorization Principles

Authenticated requests should only perform operations explicitly permitted by each platform.

The integration should:

- Respect platform permissions.
- Avoid privilege escalation.
- Execute only authorized actions.
- Preserve ownership boundaries.

Authorization should follow the permissions established by each system.

Data Protection

Business information exchanged between systems should remain protected throughout the synchronization lifecycle.

General principles include:

- Protect confidential business information.
- Minimize unnecessary data exposure.
- Synchronize only required information.
- Preserve data ownership.
- Prevent unauthorized modification.

Data Integrity

The integration should preserve the integrity of synchronized information.

General principles include:

- Prevent unintended modifications.
- Avoid partial updates whenever possible.
- Preserve business consistency.
- Detect invalid synchronization attempts.
- Maintain deterministic processing.

Integrity should remain independent of implementation technology.

Confidentiality

Sensitive information should only be accessible to authorized systems and users.

Examples include:

- Authentication credentials
- Customer information
- Internal identifiers
- Communication records
- Business metadata

Confidential information should not be unnecessarily exposed during processing.

Operational Security

Operational activities should support secure execution.

Examples include:

- Secure credential management
- Controlled administrative access
- Secure deployment practices
- Controlled configuration changes
- Separation of development and production environments

Operational procedures are defined separately.

Logging and Auditing

Security events should be traceable through operational records.

Examples include:

- Authentication failures
- Authorization failures
- Configuration changes
- Critical processing failures
- Administrative operations

Detailed logging requirements are defined separately.

Security Risks

The integration should consider risks such as:

- Unauthorized API access
- Credential exposure
- Data tampering
- Duplicate processing
- Misconfigured permissions
- Platform compromise
- Service interruption

Risk mitigation strategies are defined during implementation.

Security Constraints

The following constraints apply.

- Security must not modify business ownership.
- Security controls should remain independent of business logic.
- Security should not reduce data integrity.
- Security measures should remain consistent across all integration flows.
- Sensitive information should be protected throughout the synchronization lifecycle.

Design Principles

The security model follows these principles:

- Confidentiality
- Integrity
- Availability
- Least Privilege
- Defense in Depth
- Accountability
- Traceability
- Maintainability

Future Detailed Security Specification

The implementation phase will define detailed security specifications including:

- Authentication Methods
- Authorization Model
- Credential Management
- Secret Rotation
- Encryption Requirements
- Access Control Policies
- Audit Requirements
- Security Incident Procedures
- Compliance Considerations

This document serves as the conceptual foundation for those implementation specifications.

Related Documents

This document should be read together with:

- Error Handling Strategy
- Retry Strategy
- Logging Strategy
- Monitoring Strategy
- Validation Rules
- Integration Flows Design